



Σχεδιασμός Ενσωματωμένων Συστημάτων

9ο εξάμηνο, Ακαδημαϊκή περίοδος 2025-2026

4η Εργαστηριακή Άσκηση

Αγγελική Ζέρβα: 03121101

Δημήτρης Καμπανάκης: 03121012

Περιεχόμενα

Εισαγωγή	3
Άσκηση 1	3
Ερώτημα Α	3
Ερώτημα Β	3
Ερώτημα Γ	4
Ερώτημα Δ	12
Άσκηση 2	13
Ερώτημα Α	13
Ερώτημα Β	14

Ερώτημα Γ	16
Κώδικας για optimized design	17

Εισαγωγή

Στην άσκηση αυτή θα εξοικειωθούμε στον προγραμματισμό Field Programmable Gate Arrays (FPGA) με High Level Synthesis (HLS). Η άσκηση αυτή περιλαμβάνει τον κώδικα για μια εφαρμογή με GANs (Generative Adversarial Networks), η οποία ανακασκευάζει ημιτελείς εικόνες από χειρόγραφα ψηφία. Συγκεκριμένα οι εικόνες που χρησιμοποιούμε προέρχονται από το MNIST dataset. Ο σκοπός μας στην άσκηση είναι να μεγιστοποιήσουμε το speedup στην hardware function χρησιμοποιώντας τα κατάλληλα HLS optimizations και να μερήσουμε την ποιότητα ανακατασκευής των εικόνων.

Άσκηση 1

Ερώτημα Α

Στην συνάρτηση `forward_propagation` υπάρχουν τα layers που τρέχουν το ένα μετά το άλλο για να παράξουν το αποτέλεσμα του δικτύου, το οποίο είναι το κάτω μισό της εικόνας. Αρχικά, την θέτουμε ως hardware function για να κάνουμε estimate performance:

Details

Performance estimates for 'forward_propagation in main.cp ...

HW accelerated (Estimated cycles)	683780
-----------------------------------	--------

Resource utilization estimates for HW functions

Resource	Used	Total	% Utilization
DSP	3	80	3.75
BRAM	16	60	26.67
LUT	1760	17600	10
FF	892	35200	2.53

Figure 1: Performance unoptimized

Από το παραπάνω performance estimation μπορούμε να δούμε ότι οι κύκλοι της hardware function είναι υψηλοί, ωστόσο τα estimated resources είναι σχετικά λίγα. Θα τα αξιοποιήσουμε, λοιπόν, παρακάτω για τα κατάλληλα optimizations.

Ερώτημα Β

Παράγουμε την `sd_card` με το bitstream για την unoptimized εφαρμογή και τρέχουμε στο zybo:

```
COM8 - PuTTY
No lease, forking to background
done.
Starting Dropbear SSH server: Generating key, this may take a while...
Public key portion is:
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQAC7dQH/SnHLFG3YfcBz9FQu9dB+ytErSrZcROef57YR
I/ITake5F/RVcGYnioEkmXIKyUp91NtIYX1+hFoS38KPtRIW8svor0nN2cGJX36QIRVW1tbXH1Kxe6+
I4KvP20SqIRMolog6NAkbXzBT0x7ix8AYk8ZXGgGqHrf2KwLPLq4ZUEfkuTGP914y0xkImPhNgKRCd5w1
C3bBLsO3yAuDzkxpQzO7jTERJQcrHk56DuVE/Wl+j6q4jPGSjQXK3vEaaa/Th2/3yTsKZy4YuuyRp4c4
nYzrEippUi50OIlWRmhf89ZmhpYj5OKs7/q3LNPWJJsrCXT1Tb2KyfH72QZ root@Avnet-Digilent
-ZedBoard-2016_3
Fingerprint: md5 29:d6:32:c1:9b:75:66:2e:7a:26:00:f9:72:53:cf:46
dropbear.
hwclock: can't open '/dev/misc/rtc': No such file or directory
Starting tcf-agent: OK

sh-4.3# cd mnt
sh-4.3# ls
BOOT.BIN                System Volume Information  image.ub
GAN_ask1.elf             _sds
README.txt              data.txt
sh-4.3# ./GAN_ask1.elf
Starting dataset parsing...
Parsing finished...
Starting hardware calculations...
Hardware calculations finished.
Starting software calculations...
Software calculations finished.
Hardware cycles : 682929
Software cycles : 1473395
Speed-Up       : 2.15746
Saving results to output.txt...
sh-4.3#
```

Figure 2: Zybo run unoptimized

Παρατηρούμε ότι έχουμε 682929 hardware cycles, οι οποίοι είναι αρκετά κοντά με το estimation που έχουμε στο προηγούμενο ερώτημα. Το speedup σε σχέση με SW εκτέλεση στον ARM είναι 2.15746.

Ερώτημα Γ

Στην συνέχεια θα κάνουμε design space exporation για να βρούμε τα κατάλληλα optimizations που επιταχύνουν τον αλγόριθμο. Παρακάτω παρουσιάζουμε με την σειρά τις αλλαγές που δοκιμάσαμε και τα αντίστοιχα αποτελέσματα:

- Pipelining

Το 1ο optimization που δοκιμάζουμε είναι με χρήση του:

```
#pragma HLS PIPELINE II=1
```

να κάνουμε όλα τα loops pipelined. Σημειώνουμε ότι σε 1η φάση δοκιμάσαμε σε όσα loops είναι nested να κάνουμε pipeline το εσωτερικό. Έχουμε τα παρακάτω αποτελέσματα:

Details

Performance estimates for 'forward_propagation in main.cp ...

HW accelerated (Estimated cycles)	221487
-----------------------------------	--------

Resource utilization estimates for HW functions

Resource	Used	Total	% Utilization
DSP	4	80	5
BRAM	16	60	26.67
LUT	1897	17600	10.78
FF	881	35200	2.5

Figure 3: Results for pipelining

Detail

Instance

Loop

Loop Name	Latency		Iteration Latency	Initiation Interval		Trip Count	Pipelined
	min	max		achieved	target		
- read_input	395	395	5	1	1	392	yes
- layer_1_L	11761	11761	3	1	1	11760	yes
- layer_1_act	30	30	2	1	1	30	yes
- layer_2	1700	1700	34	-	-	50	no
+ layer_2.1	31	31	3	1	1	30	yes
- layer_3_L	19608	19608	10	1	1	19600	yes

Figure 4: Loops for pipelining

Παρατηρούμε ότι έχουμε μείωση των estimated cycles στο 1/3 περίπου, χωρίς να έχουμε ιδιαίτερη αύξηση στην χρήση resources. Για τα loops παρατηρούμε ότι όλα τα loops φαίνεται να είναι pipelined εκτός από το layer 2 στο οποίο φαίνεται ότι δεν έχουμε pipelined design. Παράλληλα, δεν παρατηρούμε βετλίωση και για το layer 3.

- Pipelining outer loops

Το επόμενο μας βήμα είναι να δοκιμάσουμε αν θα υπάρχει διαφορά στα loops τα οποία είναι διπλά να κάνουμε pipeline το εξωτερικό και έχουμε τα παρακάτω αποτελέσματα:

Details

Performance estimates for 'forward_propagation in main.cp ...

HW accelerated (Estimated cycles)	86183
-----------------------------------	-------

Resource utilization estimates for HW functions

Resource	Used	Total	% Utilization
DSP	80	80	100
BRAM	41	60	68.33
LUT	6037	17600	34.3
FF	6754	35200	19.19

Figure 5: Results for outer loops pipelining

Detail

Instance

Loop

Loop Name	Latency		Iteration Latency	Initiation Interval		Trip Count	Pipelined
	min	max		achieved	target		
- read_input	395	395	5	1	1	392	yes
- layer_1	11760	11760	30	30	1	392	yes
- layer_1_act	30	30	2	1	1	30	yes
- layer_2	52	52	4	1	1	50	yes
- layer_3	400	400	10	1	1	392	yes

Figure 6: Loops for outer loop pipelining

Παρατηρούμε ότι υπάρχει πολύ σημαντική βελτίωση στα loops των layers 2 και 3, με το layer 2 κυρίως να είναι pipelined ενώ πριν δεν ήταν και το layer 3 να έχει καλύτερο συνολικό latency. Παρατηρούμε ότι η αλλαγή μας έχει ως αποτέλεσμα σημαντική μείωση σε estimated cycles, με αύξηση των resources που χρησιμοποιούνται. Ειδικά για τα DSP παρατηρούμε ότι υπάρχει χρήση 100% των διαθέσιμων, κάτι το οποίο προκύπτει ως συνδυασμός του design μας και του directive που περιορίζει την εφαρμογή μας να μην περάσει τον μέγιστο διαθέσιμο αριθμό DSP.

Στην εικόνα των loops μπορούμε να δούμε για το 1ο loop ότι με την προσέγγισή μας σε αυτό το βήμα έχουμε "χαλάσει" το pipeline, καθώς σε αντίθεση με τα loops 2 και 3 δεν βοηθάει η προσέγγισή μας. Αυτό γίνεται γιατί όλα τα outer loops σε αυτό το layer πρέπει να γράφουν σε ίδιες θέσεις μνήμης και έτσι υπάρχει ένα WAW dependency, με αποτέλεσμα να μην μπορούμε να πετύχουμε full pipeline.

- Pipeline + Unroll

Το επόμενο μας βήμα είναι να προσπαθήσουμε να διορθώσουμε το πρόβλημα που δημιουργήθηκε από την παραπάνω αλλαγή μας. Εφόσον το πρόβλημα εμφανίστηκε στο layer 1 εκεί θα εστιάσουμε την αλλαγή μας. Κρατάμε το εξωτερικό loop pipelined και θα κάνουμε το εσωτερικό full unroll, κάνοντας αντίστοιχα full array partition τον πίνακα layer_1_out. Χρησιμοποιούμε τα directives:

```
#pragma HLS ARRAY_PARTITION variable=layer_1_out complete

#pragma HLS unroll
```

Παρατηρούμε τα παρακάτω αποτελέσματα:

Details

Performance estimates for 'forward_propagation in main.cp ...

HW accelerated (Estimated cycles)	12200
-----------------------------------	-------

Resource utilization estimates for HW functions

Resource	Used	Total	% Utilization
DSP	80	80	100
BRAM	41	60	68.33
LUT	6365	17600	36.16
FF	6407	35200	18.2

Figure 7: Results for layer 1 optimization

Detail

Instance

Loop

Loop Name	Latency		Iteration Latency	Initiation Interval		Trip Count	Pipelined
	min	max		achieved	target		
- read_input	395	395	5	1	1	392	yes
- layer_1	393	393	3	1	1	392	yes
- layer_1_act	30	30	1	1	1	30	yes
- layer_2	52	52	4	1	1	50	yes
- layer_3	400	400	10	1	1	392	yes

Figure 8: Loops for layer 1 optimization

Παρατηρούμε και πάλι σε σχέση με την προηγούμενη προσέγγιση σημαντική βελτίωση στους estimated cycles της εκτέλεσης. Για τα loops μπορούμε να δούμε ότι αλλαγή υπάρχει μόνο για το layer 1 όπως είναι φυσικό. Για το layer αυτό παρατηρούμε ότι τώρα έχουμε $\Pi = 1$ και άρα έχουμε πλήρως pipelined design. Μάλιστα, παρατηρούμε και μεγάλη διαφορά στο latency που θα ευθύνεται και για την διαφορά στην ταχύτητα εκτέλεσης.

- Pipelining / Unroll for all nested loops

Εφόσον παρατηρήσαμε ότι για το layer 1 είχαμε βελτίωση για pipelining του εξωτερικού loop με unrolling του εσωτερικού, θα δοκιμάσουμε την ίδια προσέγγιση και για τα υπόλοιπα layers. Φυσικά, πραγματοποιούμε πάλι array partition, αυτήν την φορά για το layer_2.out.

Details

Performance estimates for 'forward_propagation in main.cp ...

HW accelerated (Estimated cycles)	12038
-----------------------------------	-------

Resource utilization estimates for HW functions

Resource	Used	Total	% Utilization
DSP	80	80	100
BRAM	41	60	68.33
LUT	6213	17600	35.3
FF	6349	35200	18.04

Figure 9: Results for layers 2-3 optimization

Detail

Instance

Loop

Loop Name	Latency		Iteration Latency	Initiation Interval		Trip Count	Pipelined
	min	max		achieved	target		
- read_input	395	395	5	1	1	392	yes
- layer_1	393	393	3	1	1	392	yes
- layer_1_act	30	30	1	1	1	30	yes
- layer_2	52	52	4	1	1	50	yes
- layer_3	400	400	10	1	1	392	yes

Figure 10: Loops for layers 2-3 optimization

Παρατηρούμε μια μικρή βελτίωση στους estimated cycles με λίγο αυξημένη χρήση LUTs.

- Full unroll layer_1_act

Στο επόμενο βήμα μας δοκιμάζουμε να κάνουμε πλήρως unroll το loop layer_1_act:

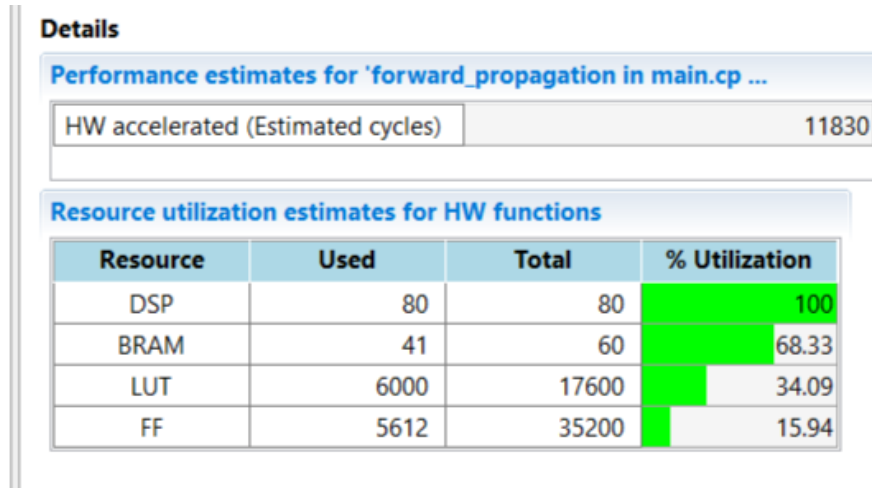


Figure 11: Results for layer_act.1 unroll

Loop

Loop Name	Latency		Iteration Latency	Initiation Interval		Trip Count	Pipelined
	min	max		achieved	target		
- read_input	395	395	5	1	1	392	yes
- layer_1	393	393	3	1	1	392	yes
- layer_2	52	52	4	1	1	50	yes
- layer_3	400	400	10	1	1	392	yes

Figure 12: Loops for layer_act.1 unroll

Παρατηρούμε ότι έχουμε μια μικρή βελτίωση στους estimated cycles, με μικρή μείωση ακόμα στην χρήση των LUTs.

- Failed unroll

Μια προσπάθεια μας που δεν φαίνεται να έχει αποτέλεσμα είναι να κάνουμε unroll και το read_input loop, όμως λόγω του μεγέθους του κυρίως παρατηρούμε ότι δεν επαρκούν οι διαθέσιμοι πόροι:

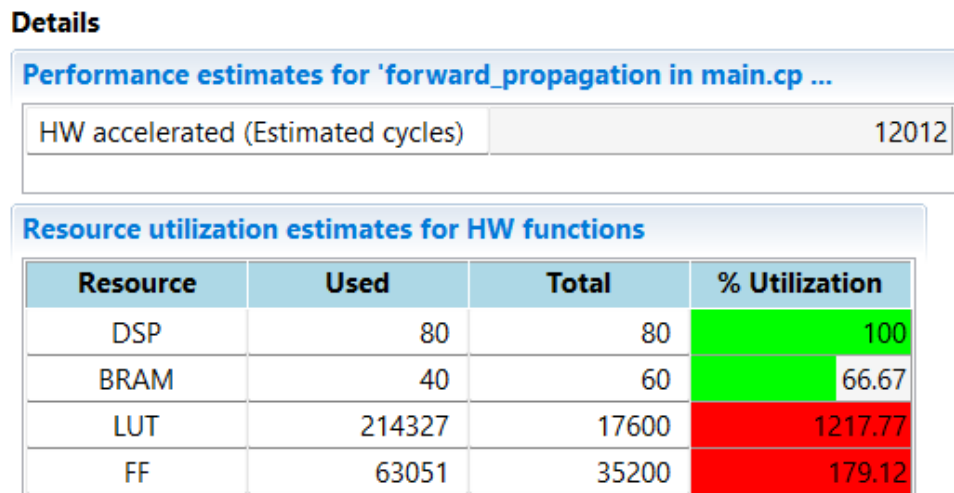


Figure 13: Failed unroll attempt

- Combined pipeline - unroll

Στο επόμενο μας βήμα, εφόσον θέλουμε να πετύχουμε όσο το δυνατόν μεγαλύτερο speedup και παρατηρούμε ότι έχουμε τους διαθέσιμους πόρους για να πετύχουμε επιπλέον βελτιώσεις στο layer 1 δοκιμάζουμε στο layer 1 να κάνουμε στο εξωτερικό loop τόσο pipelining όσο και unrolling με factor 8 (για μεγαλύτερο δεν παρατηρείται σημαντική βελτίωση σε σχέση με τα resources που καταναλώνονται). Τόσο για την προσέγγισή μας σε αυτό το βήμα, όσο και για τις παρακάτω προσεγγίσεις μας έχουμε κάνει τα αντίστοιχα array partitions στους πίνακες W1, W2 και W3, για να πετύχουμε το καλύτερο αποτέλεσμα μέσω του unrolling (το array partition γίνεται στην διάσταση που αφορά την μεταβλητή του εσωτερικού loop για κάθε πίνακα, δηλαδή dim = 2 για W1 και dim = 1 για W2 και W3).

Details

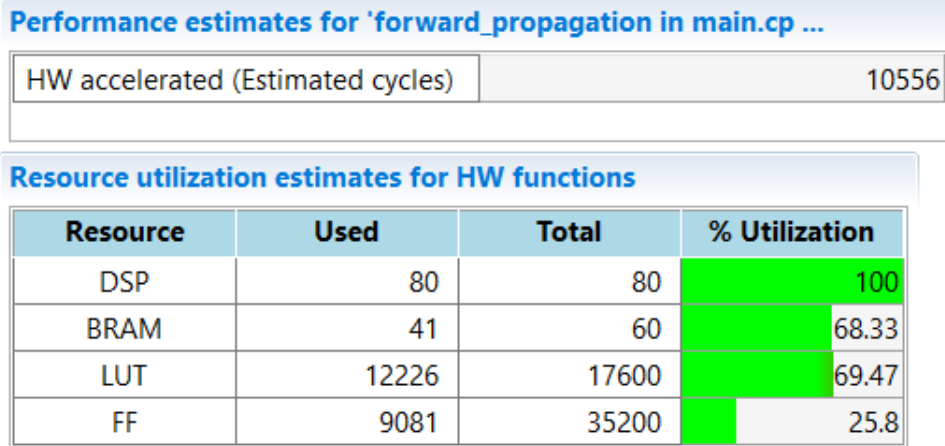


Figure 14: Combined pipelined - unroll results

Loop

Loop Name	Latency		Iteration Latency	Initiation Interval		Trip Count	Pipelined
	min	max		achieved	target		
- read_input	395	395	5	1	1	392	yes
- layer_1	197	197	6	4	1	49	yes
- layer_2	52	52	4	1	1	50	yes
- layer_3	400	400	10	1	1	392	yes

Figure 15: Combined pipelined - unroll results for loops

Παρατηρούμε ότι το design μας δεν είναι pipelined, ωστόσο έχουμε σημαντική χρονική βελτίωση με αυτήν την αλλαγή.

- Final Optimized

Σε αυτό το σημείο, εφόσον είδαμε ότι η προηγούμενή μας προσέγγιση οδήγησε σε ταχύτερη εκτέλεση πειραματιστήκαμε με συνδυασμό pipelining και unrolling για όλα τα διπλά for loops της εφαρμογής μας. Ωστόσο, για το layer 3 παρατηρήσαμε ότι ακόμα και με μικρό factor δεν είχαμε βελτίωση χρονικά και είχαμε πολύ υψηλό αριθμό χρήσης των resources. Μετά από πειράματα που τρέξαμε καταλήξαμε ότι οι καλύτεροι συνδυασμοί είναι unroll με factor 4 στα loops των layers 1 και 2 και με factor 8 μόνο στο layer 1 (τρέχουμε και τα δύο στο zybo παρακάτω). Τέλος, κάναμε array partition και τον πίνακα xbuf εφόσον υπήρχαν διαθέσιμα resources. Ωστόσο, όταν προσπαθήσαμε να παράξουμε το bitstream για να δοκιμάσουμε και αυτό το design στο zybo είχαμε μήνυμα σφάλματος, καθώς τα resources του zybo δεν

θα επαρκούσαν τελικά και για αυτό παραμείναμε στα προηγούμενα designs, τα οποία και δοκιμάζουμε παρακάτω.

Σημειώνεται ότι στο τέλος της αναφοράς έχουμε συμπεριλάβει και τον κώδικα για το τελικό optimized design της συνάρτησης forward_propagation.

```
sh-4.3# cd mnt
sh-4.3# ./GAN_ask1.elf
Starting dataset parsing...
Parsing finished...
Starting hardware calculations...
Hardware calculations finished.
Starting software calculations...
Software calculations finished.
Hardware cycles : 10779
Software cycles : 1473677
Speed-Up       : 136.717
Saving results to output.txt...
sh-4.3#
```

Figure 16: Final optimized with 8-0 unroll

```
sh-4.3# ./GAN_ask1.elf
Starting dataset parsing...
Parsing finished...
Starting hardware calculations...
Hardware calculations finished.
Starting software calculations...
Software calculations finished.
Hardware cycles : 10662
Software cycles : 1473630
Speed-Up       : 138.213
Saving results to output.txt...
sh-4.3#
```

Figure 17: Final optimized with 4-4 unroll

Παρατηρούμε ότι από τις δοκιμές μας για το unroll των εξωτερικών loop των layer 1 και 2 μαζί με pipeline η επιλογή για factors 4 και για τα δύο layers είναι η καλύτερη. Το τελικό speedup με το καλύτερο design μας είναι **138.213** και έχουμε 10662 κύκλους (για estimation όπως φαίνεται παρακάτω με 10400).

Σε σχέση με το αρχικό μας design έχουμε σημαντική μείωση στους κύκλους από το αρχικό - unoptimized, αφού αρχικά είχαμε 683929, δηλαδή έχουμε μείωση κατά 98.44% για τους κύκλους που απαιτούνται.

Ερώτημα Δ

Για το παραπάνω βελτιστοποιημένο design παρουσιάζουμε τα αποτελέσματα για το resource utilization και estimation, για τα loops και τέλος τις πληροφορίες που αφορούν τα είδη των μαθηματικών εκφράσεων για το design μας:

Details

Performance estimates for 'forward_propagation in main.cp ...

HW accelerated (Estimated cycles)	10400
-----------------------------------	-------

Resource utilization estimates for HW functions

Resource	Used	Total	% Utilization
DSP	80	80	100
BRAM	41	60	68.33
LUT	12745	17600	72.41
FF	10848	35200	30.82

Figure 18: Resources - Optimization

Loop

Loop Name	Latency		Iteration Latency	Initiation Interval		Trip Count	Pipelined
	min	max		achieved	target		
- read_input	395	395	5	1	1	392	yes
- layer_1	197	197	4	2	1	98	yes
- layer_2	28	28	5	2	1	12	yes
- layer_3	400	400	10	1	1	392	yes

Figure 19: Loops

	BRAM	DSP	FF	LUT	Bits P0	Bits P1	Bits P2	Banks/Depth	Words	W*Bits*Banks
▼ forward_propagation	82	80	10848	12554						
> I/O Ports(2)					64					
> Instances(2)	0	0	268	677						
> Memories(112)	82		546	232	1016			112	34276	310594
▼ Σ Expressions(902)	0	80	0	8054	10089	9895	1204			
> • -	0	0	0	78	16	78	0			
> • *	0	80	0	0	1332	1220	0			
> • +	0	0	0	6269	8238	8204	0			
> • and	0	0	0	5	5	5	0			
> • ashr	0	0	0	161	54	54	0			
> • icmp	0	0	0	83	226	69	0			
> • or	0	0	0	134	98	22	0			
> • select	0	0	0	1215	75	194	1204			
> • shl	0	0	0	88	32	32	0			
> • xor	0	0	0	21	13	17	0			
> Registers(974)			10034		10885					
Channels(0)	0		0	0	0			0	0	0
> Multiplexers(296)	0		0	3591	3586			0		

Figure 20: Expressions

Παρατηρούμε, όπως έχουμε επισημάνει και στο προηγούμενο ερώτημα ότι το estimation για τους κύκλους είναι κοντά στον πραγματικό αριθμό και τα resources επαρκούν για το design μας και συγκεκριμένα είναι κοντά στο όριο των resources που μπορούμε να χρησιμοποιήσουμε, καθώς δοκιμάζοντας περισσότερες αλλαγές είτε είδαμε ότι δεν επαρκούν τα estimated resources είτε επαρκούσαν οριακά στο estimation και στην πραγματικότητα δεν είχαμε αρκετά.

Για τα loops παρατηρούμε σημαντική μείωση στο latency και το trip count για κάθε loop που αφορά τα layers. Για το pipelining του design μας μπορούμε να δούμε ότι τα loops στα οποία έχουμε κάνει unroll και το εξωτερικό loop δεν είναι πλήρως pipelined, ωστόσο επειδή με πλήρως pipelined design και $\Pi = 1$ έχουμε περισσότερους κύκλους εκτέλεσης και χειρότερο speedup επιλέξαμε αυτό το design ως βέλτιστο.

Από τα αποτελέσματα για τα expressions μπορούμε να παρατηρήσουμε ότι το expression "*" του πολλαπλασιασμού χρειάζεται τα περισσότερα DSP, δηλαδή 80 που είναι και ο μέγιστος αριθμός διαθέσιμος. Όλα τα υπόλοιπα expressions φαίνεται να χρησιμοποιούν μόνο LUTs, μπορούμε να συμπεράνουμε, λοιπόν, ότι η πράξη του πολλαπλασιασμού όπως περιμέναμε είναι υπολογιστικά ακριβή και ουσιαστικά η παραπάνω διαδικασία βελτιστοποίησης του design αφορά σε μεγάλο βαθμό αυτό το expression.

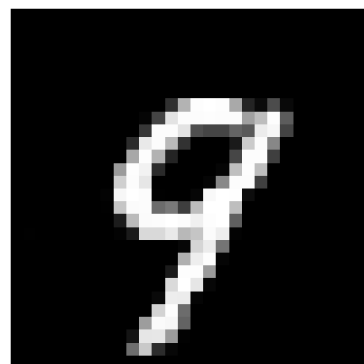
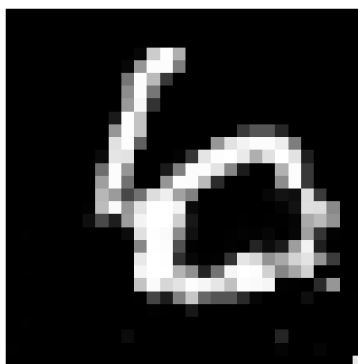
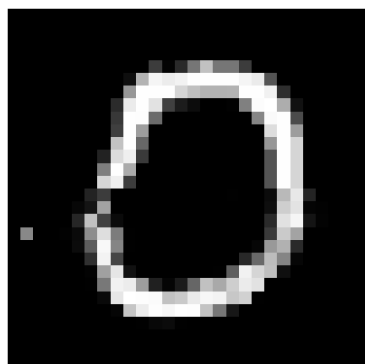
Άσκηση 2

Ερώτημα Α

Στο 2ο μέρος της εργαστηριακής άσκησης θα μετρήσουμε και θα συγκρίνουμε την ποιότητα ανακατασκευής των εικόνων. Για την διαδικασία αυτήν χρησιμοποιούμε το jupyter notebook που έχει δοθεί στο υλικό της άσκησης. Στην προηγούμενη άσκηση έχουμε τρέξει στο zybo τον αλγόριθμο του generator και λαμβάνουμε ως αποτέλεσμα ένα αρχείο output.txt, το οποίο μαζί με το data.txt θα ανεβάσουμε στο google colab για να εξάγουμε συμπεράσματα για ποιότητα ανακατασκευής των εικόνων. Για το πρόγραμμα που έχουμε τρέξει μέχρι τώρα έχουμε datatype με 8 bit δεκαδική ακρίβεια (και exp bits 1024).

Παράγουμε τα παρακάτω αποτελέσματα για τις combined εικόνες για idx = 10, 11 και 12:

Software Outputs



Hardware Outputs

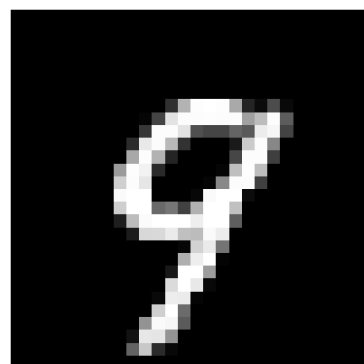
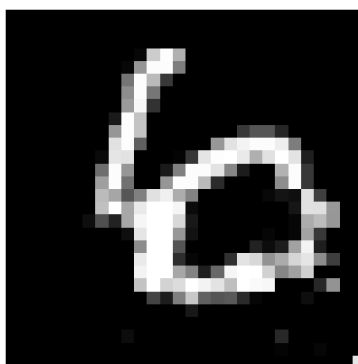
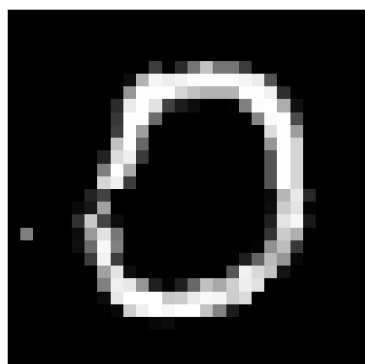


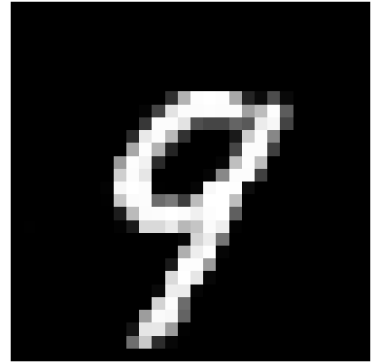
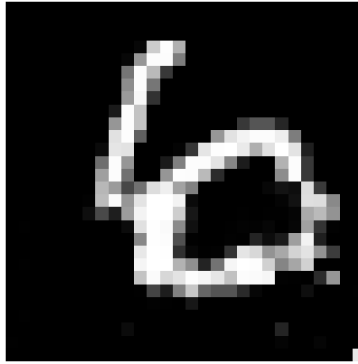
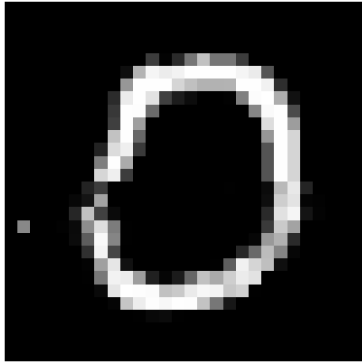
Figure 21: Outputs for 8 bit datatypes

Τα αποτελέσματα για την ποιότητα ανακατασκευής και για αυτό το βήμα θα τα σχολιάσουμε στο επόμενο ερώτημα.

Ερώτημα Β

Στην συνέχεια έχουμε πειραματιστεί με τον αριθμό των bits (και των exp bits) των datatypes που χρησιμοποιούμε. Παράγουμε ένα design με 4 bits (και exponent bits 64) και ένα με 10 bits (και 4096 exp bits αντίστοιχα). Παρουσιάζουμε τα αποτελέσματα που προκύπτουν από την επεξεργασία του output.txt με το data.txt από το jupyter notebook που μας έχει δοθεί:

Software Outputs



Hardware Outputs

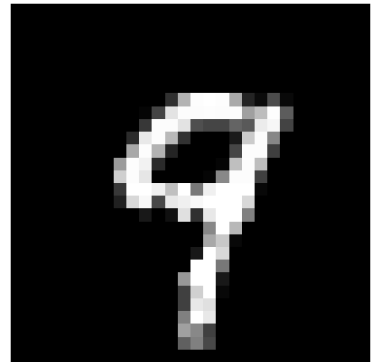
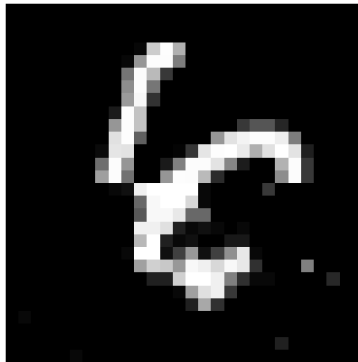
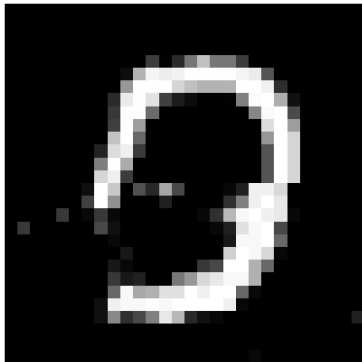


Figure 22: Outputs for 4 bit datatypes

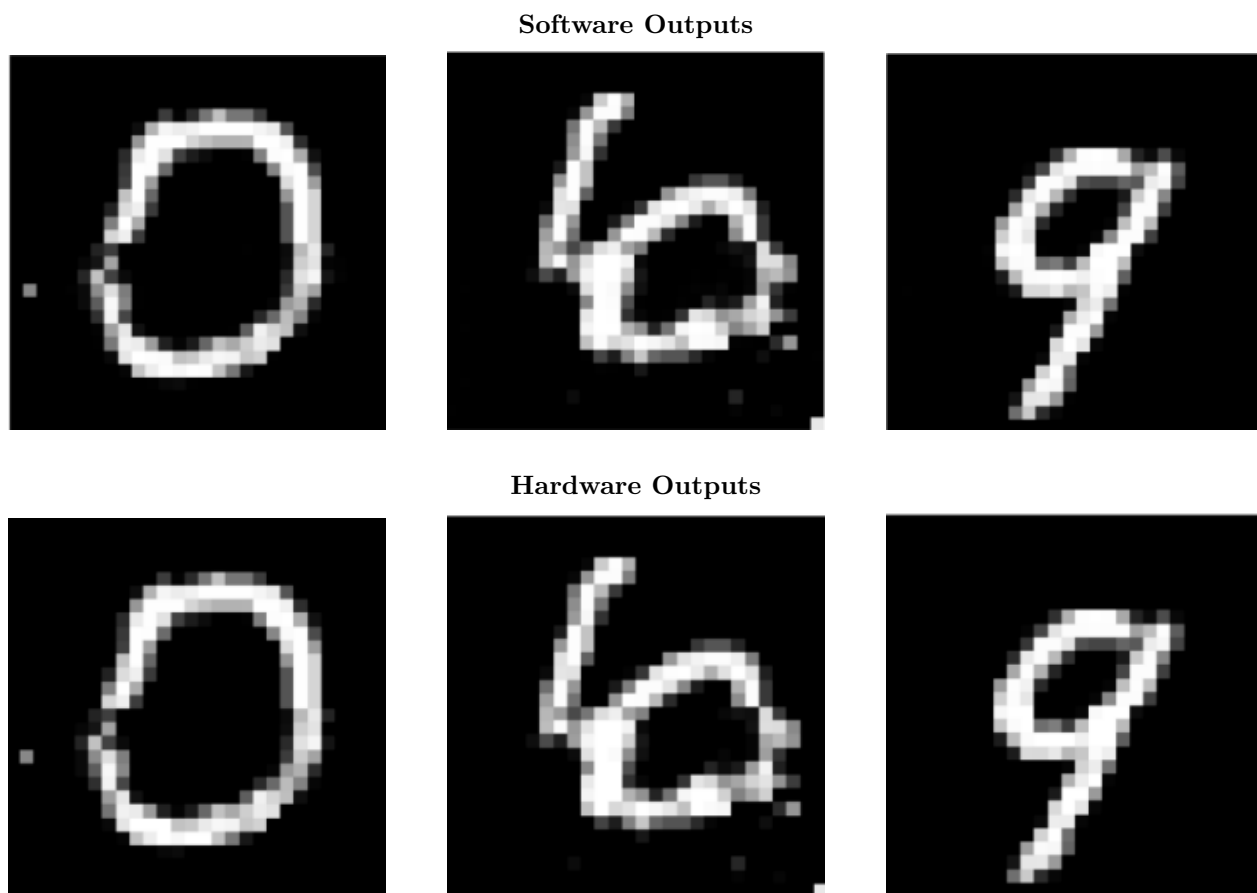


Figure 23: Outputs for 10 bit datatypes

Έχουμε ως αρχικό μέτρο σύγκρισης τα αποτελέσματα των εικόνων για τα software και hardware outputs με 8 bits ακρίβεια, για την οποία παρατηρούμε οπτικά ότι είναι αρκετά καλής ποιότητας στο hardware σχετικά με το software. Για 4 bits ακρίβεια παρατηρούμε σημαντική μείωση της ποιότητας ανακατασκευής και πλέον δεν είναι και τόσο καλά διακριτό ποιο ψηφίο είναι πραγματικά που έχει ανακατασκευαστεί. Για ακρίβεια 10 bits παρατηρούμε πάλι πολύ καλή ποιότητα στις εικόνες που προκύπτουν. Είναι λογικό αυτό το αποτέλεσμα, καθώς όσο αυξάνονται τα bits αυξάνεται η ακρίβεια για τις μεταβλητές και τις πράξεις που υλοποιούνται στο design μας.

Ερώτημα Γ

Από τα αποτελέσματα στο jupyter notebook μπορούμε να εξάγουμε και το max pixel error και το psnr για κάθε δοκιμή που πραγματοποιούμε. Στους παρακάτω πίνακες φαίνονται τα αποτελέσματα για κάθε αριθμό bits:

idx	10	11	12
max pixel error	16	17	13
PSNR	42.68	42.56	47.06

Table 1: metrics for 8-bit datatypes

idx	10	11	12
max pixel error	255	249	255
PSNR	14.05	14.63	13.52

Table 2: metrics for 4-bit datatypes

idx	10	11	12
max pixel error	5	5	4
PSNR	54.08	52.55	53.76

Table 3: metrics for 10-bit datatypes

Για τα αποτελέσματα παρατηρούμε ότι για 4 bits ακρίβεια έχουμε πολύ υψηλό max pixel error και χαμηλό PSNR. Συγκεκριμένα, για το max pixel error πετυχαίνουμε την μέγιστη τιμή. Για 8 bits παρατηρούμε σημαντική βελτίωση και στις 2 μετρικές και στην συνέχεια για 10 bits αν και παρατηρούμε βελτίωση σε σχέση με το design με τα 8 bits, αυτή δεν είναι τόσο σημαντική όσο από τα 4 bits.

Σημειώνουμε ότι το max pixel error είναι μια μετρική που για όλα τα pixel της εικόνας βρίσκει την μέγιστη διαφορά στις τιμές μεταξύ δύο pixel. Αντίθετα το PSNR είναι μετρική που αφορά τον λόγο του συνολικού σήματος (εικόνα εδώ) προς τον θόρυβο και χρησιμοποιεί στον υπολογισμό του όλα τα pixel της εικόνας. Το καλύτερο design, φυσικά, είναι αυτό που έχει μικρό max pixel error και μεγάλο PSNR.

Σύμφωνα με τα παραπάνω το PSNR είναι πιο αξιόπιστη μετρική για να κρίνουμε το output του design μας, αφού λαμβάνει υπόψιν του το σύνολο της εικόνας, ενώ το max pixel error είναι πολύ ευαίσθητο σε σφάλματα σε μικρό τμήμα της εικόνας, με μεγάλη διαφορά έντασης και έτσι σε ακραίες περιπτώσεις μπορεί να δώσει παραπλανητικά αποτελέσματα.

Η διαδικασία επιλογής των bit δεν είναι απλή, καθώς πρέπει να βρούμε μια ισορροπία ανάμεσα στην απαίτηση για το design που δίνει τα καλύτερα αποτελέσματα ως output ποιοτικά και του design που κάνει καλύτερη χρήση των πόρων. Όσο περισσότερα bits χρησιμοποιούμε, τόσο περισσότερους πόρους του zybo χρησιμοποιούμε και μπορούμε να πραγματοποιήσουμε και λιγότερα optimizations, οπότε πιθανόν να μειωθεί και το τελικό speedup που θα είχαμε. Το κέρδος αυτού του trade-off είναι ότι για μεγαλύτερο αριθμό bits έχουμε καλύτερη ποιότητα στην τελική εικόνα. Στην περίπτωση μας θεωρούμε ότι την καλύτερη ισορροπία ανάμεσα σε ποιότητα εικόνας και χρήση πόρων προσφέρει η επιλογή των 8 bits, αφού για 4 bits η ποιότητα είναι πολύ χειρότερη και για 10 bits τόσο από τις μετρικές, όσο και κυρίως οπτικά δεν μπορούμε να παρατηρήσουμε σημαντική βελτίωση της ποιότητας.

Κώδικας για optimized design

Παρακάτω παρουσιάζεται ο κώδικας μας για το τελικό optimized design:

```
void forward_propagation(float *x, float *y)
{
    quantized_type xbuf[N1];
    l_quantized_type layer_1_out[M1];
    #pragma HLS ARRAY_PARTITION variable layer_1_out complete
    l_quantized_type layer_2_out[M2];
    #pragma HLS ARRAY_PARTITION variable layer_2_out complete
```

```

#pragma HLS ARRAY_PARTITION variable=W1 complete dim=2
#pragma HLS ARRAY_PARTITION variable=W2 complete dim=1
#pragma HLS ARRAY_PARTITION variable=W3 complete dim=1

//limit resources to max DSP number of Zybo - do not change
#pragma HLS ALLOCATION instances=mul limit=80 operation

read_input:

for (int i=0; i<N1; i++)
{
#pragma HLS PIPELINE II=1
xbuf[i] = x[i];
}

// Layer 1

layer_1:
for(int i=0; i<N1; i++)
{
#pragma HLS PIPELINE II=1
#pragma HLS unroll factor = 4
for(int j=0; j<M1; j++)
{
#pragma HLS unroll
l_quantized_type last = (i==0) ? (l_quantized_type) 0 : layer_1_out[j];
quantized_type term = xbuf[i] * W1[i][j];
layer_1_out[j] = last + term;
}
}

layer_1_act:
for(int i=0; i<M1; i++)
{
#pragma HLS unroll
layer_1_out[i] = ReLU(layer_1_out[i]);
}

// Layer 2
layer_2:
for(int i=0; i<M2; i++)
{
#pragma HLS PIPELINE II=1
#pragma HLS unroll factor = 4
l_quantized_type result = 0;
for(int j=0; j<N2; j++)
{
#pragma HLS unroll
l_quantized_type term = layer_1_out[j] * W2[j][i];
result += term;
}
layer_2_out[i] = ReLU(result);
}

// Layer 3
layer_3:
for(int i=0; i<M3; i++)
{

```

```

#pragma HLS PIPELINE II=1
l_quantized_type result = 0;
for(int j=0; j<N3; j++)
{
#pragma HLS unroll
l_quantized_type term = layer_2_out[j] * W3[j][i];
result += term;
}
y[i] = tanh(result).to_float();
}
}

```